BONUSAUFGABE 1

IoT-Sensordaten mit RabbitMQ

Python und Pika in sieben Dateien

Architektur, Verarbeitung und zwei Fehlerszenarien

Gruppenmitglieder: vor dem Vortrag ergänzen

Architektur mit zwei Exchanges

Fünf Prozesse: Producer, zwei Analysen, Alarm und Monitoring

EXCHANGE	ROUTING	ZIELQUEUE
iot.sensors (topic)	temperature / humidity	iot.environment
iot.sensors (topic)	vibration	iot.machinery
iot.events (direct)	WARNING / CRITICAL	iot.alarms
iot.events (direct)	monitor / dead	iot.monitor / iot.dlq

Topic-Bindings wie sensors.temperature.* filtern im Broker. Direct verwendet die festen Ereignisschlüssel.

JSON und fachliche Regeln

Lesbare Messwerte mit den vier geforderten Attributen

```
{
  "sensor_id": "temp-01",
  "timestamp": "2026-10-26T10:00:00Z",
  "type": "temperature",
  "value": 92.5
}
```

Temperatur
über 80 °C: WARNING
über 100 °C: CRITICAL

Feuchtigkeit und Vibration besitzen ebenfalls zwei Alarmstufen.

Grenzwerte sind Annahmen für die Simulation. Routing Key und JSON-Inhalt müssen zusammenpassen.

Zuverlässig senden und bestätigen

Brokerannahme und erfolgreiche Verarbeitung haben eigene Bestätigungen

```
channel.confirm_delivery()
pika.BasicProperties(
    delivery_mode=2)
channel.basic_publish(
    ..., mandatory=True)
```

Temperatur: 10 Sekunden

Empfangene Werte sammeln. Anzahl, Mittelwert, Minimum und Maximum publizieren.

Erst nach Fensterabschluss und Monitoringpublikation die Quellmessungen bestätigen.

Producer-Backoff nach dem ersten Versuch: höchstens fünf Retries mit 1, 2, 4, 8 und 10 Sekunden Pause.

Alarme und Monitoring

Eigene Nachrichten und getrennte Consumer

MECHANISMUS	KONFIGURATION	WIRKUNG
Alarmpriorität	Queue: max 9 Nachricht: 3 / 9	CRITICAL vor wartenden WARNING
Vorabauslieferung	Alarm-Prefetch: 1	Kein Vorrat niedriger Alarme beim Consumer
Monitoring	Direct-Key: monitor	Erfolge, Typen, Alarme, Fehler und mittlere Zeit

Eine laufende Alarmverarbeitung wird nicht unterbrochen. Monitoring liest ausschließlich RabbitMQ-Nachrichten.

DLQ und 60 Sekunden TTL

Fehlerhafte oder veraltete Nachrichten verlassen den normalen Datenfluss

```
channel.basic_nack(
    delivery_tag=tag, requeue=False)
```

01 NACK ohne Requeue nutzt iot.events mit dem Routing Key dead. Die Originalnachricht landet in iot.dlq.

02 Hauptqueues: x-message-ttl=60000. Abgelaufene wartende Nachrichten tragen x-death.reason=expired.

03 Die DLQ hat keine TTL und keinen automatischen Rückweg.

Live-Demonstration

Vier Minuten im vorbereiteten Terminal

```
python start.py
python demo.py invalid
python demo.py crash
```

01 Normalbetrieb mit Aggregation, Alarmen und Monitoring beobachten.

02 Ungültige Nachricht und tatsächlichen DLQ-Eintrag zeigen.

03 Consumer beenden und erneute Zustellung nach Neustart nachweisen.

Fehlervergleich und Grenzen

AUSLÖSER	BESTÄTIGUNG	NACHRICHTENWEG
Ungültige Nachricht	NACK, requeue=False	DLQ, keine erneute Hauptqueue-Zustellung
Consumer endet vor ACK	Weder ACK noch NACK	Erneute Zustellung an Ersatzconsumer

Nach beiden Szenarien kann das System gültige Nachrichten weiterverarbeiten.

Grenzen: einzelner Broker, Auswertungen im RAM und mögliche Duplikate zwischen Publish und ACK.